

Speech Notes: Models Section

Slides 5–8 · Target: ≈ 6 min total · ~ 1.5 min/slide average

Slide	Title	Target
5	Model Selection & Architecture Overview	≈ 2 min
6	PatchCore: Transfer Learning & Architecture	≈ 1.5 min
7	Keras CAE – Autoencoder Learning Process	≈ 1.5 min
8	Keras CAE – End-to-End Pipeline	≈ 1 min

1. Slide 5 — Model Selection & Architecture Overview

[≈ 2 min]

1.1 Opening: Entering the Models section

“Now let’s talk about the models. Given the task – unsupervised anomaly detection with zero defect data – we deliberately chose two architecturally very different approaches, and I want to walk you through why.”

Note to self: You just came off the Metrics slide. The audience has AUPIMO in mind. These two models are what AUPIMO will later score.

1.2 Left column: Convolutional Autoencoder

→ **Point at:** the CAE diagram: $x \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x} \rightarrow \Delta$

“The first model is a Convolutional Autoencoder, built completely from scratch. The idea is simple: you take an input image x , compress it through a narrow bottleneck z , and reconstruct it back as $\hat{x}$. We train exclusively on defect-free images. The model learns to encode what normal looks like. When a defective image arrives at inference, the anomaly gets projected onto the nearest normal point the decoder knows – and it reconstructs a normal version. The mismatch between input and reconstruction is our anomaly signal.”

→ **Point at:** the score formula below the diagram

“We aggregate that mismatch via Top-K pooling over the per-pixel error map:

$$s = \text{TopK}_{0.2\%}(0.84(1 - \text{SSIM}) + 0.16 \text{MAE})$$

Top-K at 0.2% means we take the average of the worst 0.2% of pixels – those are your defect candidates. I will come back to the SSIM and MAE choice when we get to the training slide.”

→ **Point at:** the three bullet points: complete control, intuition, trade-offs

“We built this from scratch precisely for the control it gives us: full freedom over the bottleneck

dimensions, the loss, the augmentation strategy. The trade-off is a real one – autoencoders are prone to blurriness or, worse, to learning identity shortcuts if we are not careful.”

1.3 Right column: PatchCore + ResNet-18

→ **Point at:** the PatchCore diagram: $x \rightarrow \text{ResNet-18 (frozen)} \rightarrow \phi(x) \rightarrow \mathcal{M} \rightarrow d^*$

“The second model is PatchCore, from Intel’s Anomalib library. Completely different paradigm: no gradient updates, no training in the traditional sense. Instead, you pass every normal training image through a frozen ResNet-18 backbone, extract mid-level patch feature vectors – that is $\phi(x)$ – and store a compressed subset in a memory bank $\mathcal{M}$.”

“At inference, the anomaly score for a test patch is just the nearest-neighbour L2 distance to the closest stored normal prototype:

$$s(p^*) = \min_{m \in \mathcal{M}} \|\phi(p^*) - m\|_2$$

If a patch has no close match in memory, it is anomalous. This is closer to database indexing than machine learning.”

→ **Point at:** coreset bullet: “Compresses the memory bank by 90%+”

“Anomalib handles coreset subsampling via greedy farthest-point sampling – the 10% of vectors that cover the whole feature space best. Our contribution was integrating this into our evaluation protocol and cross-validating it against the CAE.”

1.4 Why two such different paradigms?

“These two models fail in completely different ways – and that is precisely why we chose both. A CAE can fail silently on unseen texture distributions. PatchCore fails when memory bank coverage has gaps. Running both gives us orthogonal error signals, and PatchCore also acts as a natural upper-bound baseline: if frozen ImageNet features cannot detect a defect, no learned model will.”

Transition: “Let me go into PatchCore first, since it was actually our starting point.”

2. Slide 6 — PatchCore: Transfer Learning & Architecture [≈1.5 min]

2.1 Opening: Why PatchCore was first

“PatchCore was our first model, and the reason is straightforward: it was the most intuitive entry point into anomaly detection without any defect data. Anomalib gave us a fully working pipeline out of the box – we could evaluate it on day one, before writing a single line of custom model code. That made it an invaluable baseline and an anchor point for everything

that followed.”

2.2 Left column: Transfer Learning flowchart

→ **Point at:** *the four-step vertical flowchart*

“Walk through top to bottom.

Normal Factory Images: *zero defect labels needed, Day-1 deployable.*

Frozen Feature Extractor: *ResNet-18, pre-trained on ImageNet, weights fully locked. We use layers 2 and 3 specifically – not the final classification layer which encodes cat vs truck semantics, but the mid-level layers that encode edges, textures, and local structures. Those transfer perfectly to surface defect domains.*

Coreset Memory Bank: *greedy farthest-point sampling compresses all patch vectors down to ~10%, keeping maximum coverage of the feature space manifold.*

Inference: *nearest-neighbour distance to $\mathcal{M}$. Large distance means anomalous.”*

2.3 Right column: Results and key properties

→ **Point at:** *the four heatmap images: wood, zipper, transistor, toothbrush*

“These are PatchCore’s heatmaps across four MVTec categories. White overlay is the ground truth; green is the predicted mask at the 99th percentile threshold. The match is strong even for visually challenging defects like the liquid contamination on wood.”

→ **Point at:** *Key Features and “Why it excels” bullet lists*

“Two properties I want to highlight. First, cold-start eliminated – no defect data collection before deployment. Second, IP protection: the memory bank stores embeddings, not raw images – it is not reversible.

On why it excels: the frozen backbone is critical. If you fine-tuned ResNet on normal images only, you would collapse the embedding space to the factory distribution and lose exactly the generalisation that makes PatchCore sensitive to subtle defects.”

Transition: *“Now let us look at our custom model – the Keras CAE – and how we built it up from scratch, failure mode by failure mode.”*

3. Slide 7 — Keras CAE: Autoencoder Learning Process [≈1.5 min]

3.1 Opening: A story of incremental decisions

“This slide tells the development story of the CAE. We did not arrive at the final architecture

on the first try. Every design decision here was motivated by a concrete problem we ran into.”

3.2 Left column: The core concept

→ **Point at:** *Training Phase / Inference Phase bullets*

“The concept is simple: train on normals, fail to reconstruct defects at inference. But the how is what matters – and that is the right column.”

3.3 Right column: Key design decisions

Activation: ReLU → (LeakyReLU) → ELU —

“We started with ReLU. The dying neuron problem hit us fast: neurons in the bottleneck locked to hard zero and stopped contributing gradients entirely. On our relatively limited dataset – only 200 to 300 normal images per category – we cannot afford to lose representation capacity. With ReLU, any negative input is hard zeroed out. With ELU, the chance of hitting exact zero is virtually zero: for negative inputs, it smoothly follows $\alpha(e^x - 1)$, ensuring continuous, non-zero gradient flow so every parameter keeps learning even across small batches. We also tested Leaky ReLU, but its sharp kink at zero introduced noise into fine-grained reconstruction gradients. ELU is continuously differentiable, saturates gracefully for large negative inputs, and naturally pushes mean activations toward zero. That made it the optimal choice for our small-data regime.”

BatchNorm and bias —

“Early on we used bias terms and no BatchNorm. Once we added BatchNormalisation, we removed the bias. This is by design: BatchNorm already shifts activations through its own learnable parameter β . A separate bias is redundant – it is absorbed into β – so `use_bias=False` keeps the model clean. BatchNorm also stabilises training by reducing internal covariate shift, which on small datasets like MVTec is very valuable.”

Optimizer: Adam → AdamW —

“Standard Adam couples weight decay with the adaptive learning rate, which under-regularises on small datasets. AdamW decouples them: the gradient step and the weight decay apply independently. With only a few hundred normal training images per category, that stronger regularisation matters.”

MIM: preventing the identity shortcut — → **Point at:** *“Masked Image Modelling: 25% of patches randomly masked” bullet*

“Finally, Masked Image Modelling: we randomly zero 25% of input patches before the encoder sees them. Without this, the model can learn a trivial identity map and reconstruct anomalies just as well as normals – total failure. MIM forces it to infer missing content from context and build a genuine structural model of what normal looks like.”

Training loss vs. inference score – Literature vs. Implementation —

“Here is a technical nuance worth clarifying regarding literature versus implementation. In the seminal paper by Bergmann et al. (2019, VISAPP), they evaluated SSIM with both L_1 and L_2 , and specifically recommended $SSIM + L_1$ (MAE) for both training and evaluation.

In our project:

- During **training**, we implemented $SSIM + MSE$ (L_2):

$$\mathcal{L}_{train} = 0.84(1 - SSIM) + 0.16 MSE$$

MSE provided smooth, stable gradient convergence in our Keras pipeline.

- At **inference**, our pixel anomaly map strictly aligns with Bergmann’s formula, using $SSIM + MAE$ (L_1):

$$Score = \text{TopK}_{0.2\%}(0.84(1 - SSIM) + 0.16 MAE)$$

Why use MAE for inference? Because squaring errors with L_2 at test time penalises large errors quadratically, blowing out bright spots and distorting localization heatmaps. Linear L_1 error preserves the true proportional contrast of defects.

”

► **Watch out:** If an examiner asks: “Didn’t Bergmann recommend $SSIM + L_1$ for both?”
Answer: “Yes, absolutely! Bergmann et al. (2019) found $SSIM + L_1$ best overall. In our Keras training implementation, we coupled SSIM with MSE (L_2) because it offered smooth gradient convergence during training, but for inference and error map generation, we stayed true to Bergmann’s linear L_1 formulation so the pixel error map isn’t distorted quadratically by squared errors.”

Transition: “Let me now show you how all these pieces come together in the full automated pipeline.”

4. Slide 8 — Keras CAE: End-to-End Pipeline

[≈1 min]

4.1 Step-by-step walkthrough

→ **Point at:** the 9-step TikZ pipeline diagram – three rows

“The pipeline runs left to right across three rows. Let me go through it quickly.”

Row 1 – Data Preparation (Steps 1–3):

“Raw MVTEC data goes through preprocessing – Otsu plus Canny edge detection for foreground masking, CLAHE contrast enhancement, and blur – then branches on category type. Textures get rotation and flip augmentations; objects get colour jitter and noise. This category-aware

branching matters because texture datasets are rotationally invariant while object datasets are not.”

Row 2 – Model Building and Training (Steps 4–6):

*“Step 4 applies Masked Image Modelling – 25% of patches zeroed. Step 5 builds the CAE with ELU, BatchNorm, AdamW. Step 6 trains with the **SSIM+MSE loss** – this is the training loss, not the anomaly score.”*

Row 3 – Inference and Evaluation (Steps 7–9):

*“Step 7 scores images: frozen weights, per-pixel **SSIM+MAE error map**, Top-K pooling to a single image-level score. Step 8 applies the adaptive threshold – calibrated on the 15% normal validation partition using quantile or Mahalanobis distance. This gives a Day-1 deployable false-alarm-rate guarantee with zero defect knowledge. Step 9 evaluates on the locked test set: AUROC and AUPIMO.”*

→ **Point at:** the Optuna caption at the bottom

“The whole pipeline is orchestrated by Optuna – Bayesian optimisation over augmentation strategy, latent dimensions, crop size – targeting Pixel AUPIMO as the objective. We target AUPIMO rather than AUROC because AUPIMO evaluates only the operationally relevant region below false-alarm rate 10^{-4} . Optimising AUROC could improve the broad curve while degrading exactly the narrow region that matters in a real factory.”

4.2 Closing the Models section

“So to summarise the Models section: PatchCore is our high-accessibility no-training baseline that operationalises frozen ImageNet features. The Keras CAE is our from-scratch model where every decision – ELU, BatchNorm, MIM, AdamW, and the loss-versus-score distinction – was driven by a concrete failure mode we needed to address. In the next section I will show you how we evaluated both under the same zero-leakage protocol.”

End of Models section speech notes.